Universidad Tecnológica del Perú

AVANCE 3 DE PROYECTO FINAL

Integración Frontend + Backend + Base de Datos

LAFORSE-HG: *Sistema web de gestión de solicitudes y casos periciales*

Curso: Marcos de Desarrollo Web (100000S52T)

Docente: Hernan Francisco Peña Carnero

N°	Apellidos	Nombres	Código
1	Colchado Chavez	Jens Fabrizio	U24270825

Lima, octubre de 2026

Índice

1. Título del proyecto **3**
2. Resumen del avance **3**
3. Tecnologías utilizadas **3**
4. Diseño de base de datos **5**
5. Servicios y lógica de negocio **8**
5.1 Métodos CRUD 8
5.2 Validaciones 8
5.3 Reglas de negocio 9
5.4 Manejo de errores 10
6. Integración frontend y backend **10**
6.1 Flujo de una operación (registrar un caso) 11
6.2 Rutas 11
7. Interfaces del sistema **12**
7.1 Listado 12
7.2 Registro 14
7.3 Edición 15
7.4 Eliminación 16
7.5 Validaciones y reglas de negocio 17
7.6 Peritos y solicitudes 19
7.7 API REST y vista móvil 22
8. Pruebas realizadas **23**
8.1 Pruebas funcionales 23
8.2 Almacenamiento en MySQL 24
8.3 Pruebas del API REST 25
8.4 Pruebas automatizadas 25
9. Trabajo pendiente **26**
10. Conclusiones **26**
11. Anexos **28**
Anexo A. Especificación de casos de uso 28
Anexo B. Diagrama de casos de uso 32
Anexo C. Diagrama de secuencia: Registrar caso 33
Anexo D. Diagrama de clases 34
Anexo E. Código fuente relevante 34
Anexo F. Ejecución del proyecto 38

1. Título del proyecto

LAFORSE-HG: Sistema web de gestión de solicitudes y casos periciales con Spring Boot, Thymeleaf y MySQL.

El título refleja el objetivo del sistema en este avance: dejar de ser un sitio informativo para convertirse en una aplicación que registra, consulta, actualiza y elimina en MySQL las solicitudes de los clientes, los casos (expedientes) y los peritos del laboratorio LAFORSE-HG.

2. Resumen del avance

En el Avance 2 la aplicación Spring Boot mostraba los datos desde listas en memoria y las solicitudes se perdían al reiniciar. En este Avance 3 se integró la capa de persistencia con **Spring Data JPA, Hibernate y MySQL 8**, y se construyó el módulo de gestión del laboratorio. Los avances concretos son:

- **Integración frontend–backend–BD:** las vistas Thymeleaf y un API REST en JSON usan los mismos servicios, que leen y escriben en la base de datos `laforse_db`.
- **Base de datos MySQL** con cuatro tablas relacionadas (`servicio`, `perito`, `solicitud` y `caso`), claves primarias, foráneas, restricciones UNIQUE y CHECK, y un script SQL de creación con datos iniciales.
- **CRUD funcional** de casos y de peritos (registrar, listar con filtros, editar y eliminar) y gestión de solicitudes (listar, descartar, eliminar y convertir en caso).
- **Validaciones** en dos niveles: HTML5/Bootstrap en el navegador y Bean Validation (`@Valid`) en el servidor, con mensajes por campo.
- **Reglas de negocio** en la capa de servicio (expediente y correo únicos, fecha no futura, casos cerrados en solo lectura, peritos con casos no eliminables, entre otras) y **manejo de errores** centralizado para vistas (página de error) y API (JSON con código HTTP).
- **Mejoras visuales:** menú «Gestión» en la barra de navegación, alertas de éxito/error, modal de confirmación de eliminación reutilizable, tarjetas de conteo por estado, búsqueda y formularios responsivos.
- **Pruebas:** 20 pruebas automatizadas con JUnit 5 y MockMvc (páginas, CRUD, reglas de negocio y API) y pruebas funcionales manuales con evidencias en pantalla y en MySQL.

3. Tecnologías utilizadas

Tabla 1. *Tecnologías del sistema.*

Capa	Tecnología	Uso en el proyecto
Backend	Java 17, Spring Boot 3.3.4	Aplicación con Tomcat embebido y configuración automática
Backend	Spring Web MVC	Controladores @Controller (vistas) y @RestController (API JSON)
Backend	Spring Data JPA + Hibernate 6	ORM: entidades @Entity, repositorios JpaRepository, consultas derivadas y JPQL

Capa	Tecnología	Uso en el proyecto
Backend	Bean Validation (Hibernate Validator)	@NotBlank, @Size, @Email, @Pattern, @PastOrPresent, @NotNull, @AssertTrue
Frontend	Thymeleaf 3.1	Plantillas del servidor: fragmentos, th:each, th:object, th:field, #fields
Frontend	HTML5, CSS3, JavaScript	Estructura semántica, estilos propios y lógica del modal de eliminación
Frontend	Bootstrap 5.3.3 + Bootstrap Icons	Grilla responsiva, navbar con dropdown, tablas, formularios, modales, alertas, badges
Base de datos	MySQL 8 + MySQL Connector/J	Base de datos relacional laforse_db (motor InnoDB)
Herramientas	Maven, NetBeans, cliente mysql	Dependencias y compilación, IDE, consultas a la BD
Herramientas	JUnit 5, MockMvc, H2 (solo pruebas), curl / Postman	Pruebas automatizadas y pruebas manuales del API

Las dependencias nuevas respecto al Avance 2 se declaran en `pom.xml`:

pom.xml (fragmento)

```
    <!-- Avance 3: ORM con Hibernate / Spring Data JPA -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <!-- Driver JDBC de MySQL 8 -->
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>
```

application.properties (conexión a MySQL)

```
# ---------- Base de datos MySQL ----------
# Cambie usuario y clave por los de su MySQL local. La BD se crea sola si no existe.
spring.datasource.url=jdbc:mysql://localhost:3306/laforse_db?
createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Americ
a/Lima
spring.datasource.username=root
spring.datasource.password=root

# ---------- JPA / Hibernate ----------
# update: crea o actualiza las tablas a partir de las entidades (@Entity)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false
```

4. Diseño de base de datos

La base de datos `laforse_db` tiene cuatro tablas. `servicio` es el catálogo del laboratorio; `perito` guarda al equipo técnico; `solicitud` registra lo que envía un cliente desde el formulario de contacto; y `caso` es el expediente pericial, que se relaciona con un perito, un servicio y, opcionalmente, con la solicitud que lo originó.

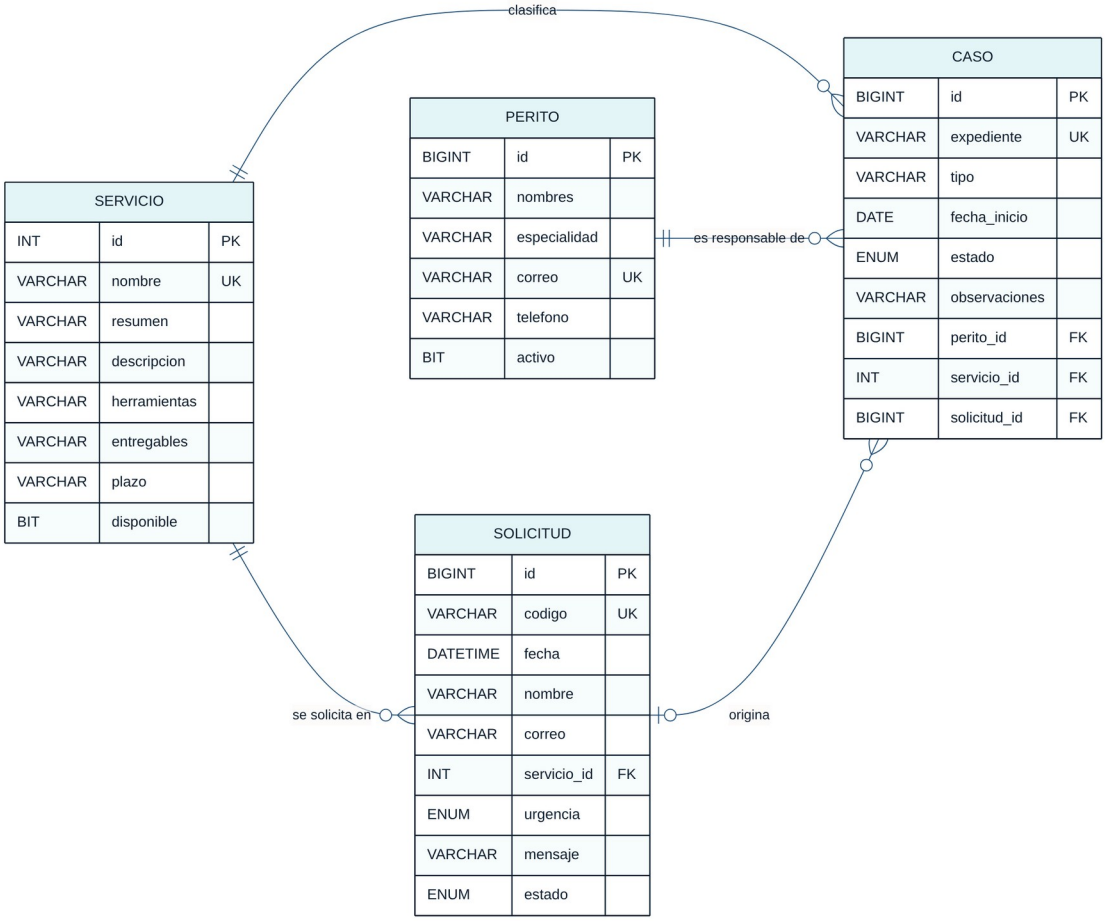


Figura 1. *Modelo entidad-relación de laforse_db.*

Tabla 2. *Tablas, claves y restricciones (todas las claves primarias son AUTO_INCREMENT).*

Tabla	Clave primaria	Claves foráneas	Restricciones
servicio	id INT	—	UNIQUE(nombre)
perito	id BIGINT	—	UNIQUE(correo); CHECK celular ^9[0-9]{8}$

Tabla	Clave primaria	Claves foráneas	Restricciones
solicitud	id BIGINT	servicio_id → servicio(id)	UNIQUE(codigo); ENUM urgencia y estado
caso	id BIGINT	perito_id → perito(id) servicio_id → servicio(id) solicitud_id → solicitud(id) (opcional)	UNIQUE(expediente); CHECK formato PJ-AAAA-NNNN; ENUM estado

Tabla 3. *Relaciones entre tablas.*

Relación	Cardinalidad	Mapeo JPA
servicio – solicitud	1 : N	@ManyToOne Servicio servicio (en Solicitud)
servicio – caso	1 : N	@ManyToOne Servicio servicio (en Caso)
perito – caso	1 : N	@ManyToOne Perito perito (en Caso)
solicitud – caso	0..1 : N	@ManyToOne Solicitud solicitud, columna opcional

El script completo (con los datos iniciales) está en `database/laforse_db.sql`. A continuación se muestra la parte de creación de tablas:

Script SQL de creación (laforse_db.sql)

```
DROP DATABASE IF EXISTS laforse_db;
CREATE DATABASE laforse_db CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish_ci;
USE laforse_db;

-- ------------------------------------------------------------------
-- Catálogo de servicios del laboratorio
-- ------------------------------------------------------------------
CREATE TABLE servicio (
  id            INT          NOT NULL AUTO_INCREMENT,
  nombre        VARCHAR(80)  NOT NULL,
  resumen       VARCHAR(200) NOT NULL,
  descripcion   VARCHAR(600) NOT NULL,
  herramientas  VARCHAR(300) NULL COMMENT 'Lista separada por ;',
  entregables   VARCHAR(300) NULL COMMENT 'Lista separada por ;',
  plazo         VARCHAR(60)  NULL,
  icono         VARCHAR(40)  NULL,
  disponible    BIT(1)       NOT NULL DEFAULT b'1',
  CONSTRAINT pk_servicio PRIMARY KEY (id),
  CONSTRAINT uk_servicio_nombre UNIQUE (nombre)
) ENGINE = InnoDB;

-- ------------------------------------------------------------------
-- Peritos informáticos
-- ------------------------------------------------------------------
```

```
CREATE TABLE perito (
  id            BIGINT       NOT NULL AUTO_INCREMENT,
  nombres       VARCHAR(80)  NOT NULL,
  especialidad  VARCHAR(60)  NOT NULL,
  correo        VARCHAR(100) NOT NULL,
  telefono      VARCHAR(9)   NULL,
  activo        BIT(1)       NOT NULL DEFAULT b'1',
  CONSTRAINT pk_perito PRIMARY KEY (id),
  CONSTRAINT uk_perito_correo UNIQUE (correo),
  CONSTRAINT ck_perito_telefono CHECK (telefono IS NULL OR telefono REGEXP '^9[0-9]{8}$')
) ENGINE = InnoDB;

-- ------------------------------------------------------------------
-- Solicitudes recibidas desde el formulario de contacto
-- ------------------------------------------------------------------
CREATE TABLE solicitud (
  id            BIGINT       NOT NULL AUTO_INCREMENT,
  codigo        VARCHAR(20)  NOT NULL,
  fecha         DATETIME(6)  NOT NULL,
  nombre        VARCHAR(80)  NOT NULL,
  correo        VARCHAR(100) NOT NULL,
  telefono      VARCHAR(9)   NULL,
  servicio_id   INT          NOT NULL,
  urgencia      ENUM('BAJA','MEDIA','ALTA','CRITICA')         NOT NULL,
  mensaje       VARCHAR(600) NOT NULL,
  estado        ENUM('PENDIENTE','ATENDIDA','DESCARTADA')     NOT NULL DEFAULT 'PENDIENTE',
  CONSTRAINT pk_solicitud PRIMARY KEY (id),
  CONSTRAINT uk_solicitud_codigo UNIQUE (codigo),
  CONSTRAINT fk_solicitud_servicio FOREIGN KEY (servicio_id) REFERENCES servicio (id)
) ENGINE = InnoDB;

-- ------------------------------------------------------------------
-- Casos (expedientes periciales)
-- ------------------------------------------------------------------
CREATE TABLE caso (
  id            BIGINT       NOT NULL AUTO_INCREMENT,
  expediente    VARCHAR(12)  NOT NULL,
  tipo          VARCHAR(120) NOT NULL,
  fecha_inicio  DATE         NOT NULL,
  estado        ENUM('DICTAMEN_ENTREGADO','EN_ANALISIS','URGENTE','PROGRAMADO') NOT NULL,
  observaciones VARCHAR(500) NULL,
  perito_id     BIGINT       NOT NULL,
  servicio_id   INT          NOT NULL,
  solicitud_id  BIGINT       NULL,
  CONSTRAINT pk_caso PRIMARY KEY (id),
  CONSTRAINT uk_caso_expediente UNIQUE (expediente),
  CONSTRAINT ck_caso_expediente CHECK (expediente REGEXP '^PJ-[0-9]{4}-[0-9]{4}$'),
  CONSTRAINT fk_caso_perito    FOREIGN KEY (perito_id)    REFERENCES perito (id),
  CONSTRAINT fk_caso_servicio  FOREIGN KEY (servicio_id)  REFERENCES servicio (id),
  CONSTRAINT fk_caso_solicitud FOREIGN KEY (solicitud_id) REFERENCES solicitud (id)
) ENGINE = InnoDB;

CREATE INDEX ix_caso_estado ON caso (estado);
```

```
CREATE INDEX ix_solicitud_estado ON solicitud (estado);
```

Además del script, la aplicación usa `spring.jpa.hibernate.ddl-auto=update`: si las tablas no existen, Hibernate las crea a partir de las entidades, y la clase `DataSeeder` carga los mismos datos iniciales solo cuando las tablas están vacías.

5. Servicios y lógica de negocio

La lógica está en la capa `service`. Los controladores solo reciben la petición, validan el formulario con `@Valid` y delegan en el servicio; los servicios aplican las reglas de negocio dentro de una transacción (`@Transactional`) y usan los repositorios de Spring Data JPA para acceder a MySQL.

5.1 Métodos CRUD

Tabla 4. *Métodos CRUD de los servicios.*

Servicio	Método	Operación	Repositorio / SQL generado
CasoService	registrar(CasoForm)	Create	existsByExpediente → save → INSERT
CasoService	listar(estado, texto)	Read	Consulta JPQL buscar(...) → SELECT con JOIN a perito
CasoService	actualizar(id, CasoForm)	Update	findById → existsByExpedienteAndIdNot → save → UPDATE
CasoService	eliminar(id)	Delete	findById → delete → DELETE
PeritoService	guardar(Perito)	Create / Update	existsByCorreoIgnoreCase[AndIdNot] → save
PeritoService	listar() / eliminar(id)	Read / Delete	findAllByOrderByNombresAsc / existsByPeritoId → delete
SolicitudService	registrar(ContactoForm)	Create	countByCodigoStartingWith → save (código SOL-AAAA-NNNN)
SolicitudService	listar / cambiarEstado / eliminar	Read / Update / Delete	findByEstadoOrderByFechaDesc / save / delete

5.2 Validaciones

Tabla 5. *Validaciones del formulario de casos y peritos.*

Campo	Anotación (servidor)	Validación en el navegador
Expediente	@NotBlank, @Pattern(^PJ-\d{4}-\d{4}$)	required, pattern="PJ-[0-9]{4}-[0-9]{4}"

Campo	Anotación (servidor)	Validación en el navegador
Tipo de peritaje	@NotBlank, @Size(5–120)	required, minlength, maxlength
Fecha de inicio	@NotNull, @PastOrPresent	type="date", max = fecha de hoy
Estado, perito, servicio	@NotNull	required en el select
Observaciones	@Size(max = 500)	maxlength="500"
Perito: nombres / correo	@NotBlank, @Size, @Email	required, type="email"
Perito / cliente: celular	@Pattern(^$\|^9\d{8}$)	pattern="9[0-9]{8}"

5.3 Reglas de negocio

- **RN01.** El número de expediente es único y sigue el formato PJ-AAAA-NNNN. El sistema sugiere el siguiente número del año.
- **RN02.** El correo de un perito es único (sin distinguir mayúsculas).
- **RN03.** Solo se asignan casos nuevos a peritos activos.
- **RN04.** Un caso con estado «Dictamen entregado» queda cerrado: no se puede editar ni eliminar (se conserva por trazabilidad).
- **RN05.** No se elimina un perito con casos asignados; se debe desactivar.
- **RN06.** Al abrir un caso desde una solicitud, esta pasa a «Atendida»; si su urgencia es «Crítica», el caso se crea como «Urgente».
- **RN07.** Una solicitud que ya generó un caso no se puede eliminar.

CasoService.java: registrar, actualizar y eliminar

```
    public Caso registrar(CasoForm form) {
        if (repo.existsByExpediente(form.getExpediente())) {
            throw new ReglaNegocioException("expediente", "El expediente " +
form.getExpediente() + " ya está registrado");
        }
        Caso caso = new Caso();
        copiar(form, caso, true);
        Caso guardado = repo.save(caso);
        // Integración: la solicitud de origen pasa a "Atendida"
        if (guardado.getSolicitud() != null) {
            solicitudService.cambiarEstado(guardado.getSolicitud().getId(),
EstadoSolicitud.ATENDIDA);
        }
        return guardado;
    }

    public Caso actualizar(Long id, CasoForm form) {
        Caso caso = obtener(id);
        if (caso.isCerrado()) {
            throw new ReglaNegocioException("El caso " + caso.getExpediente()
```

```
                + " tiene el dictamen entregado y ya no se puede modificar.");
        }
        if (repo.existsByExpedienteAndIdNot(form.getExpediente(), id)) {
            throw new ReglaNegocioException("expediente", "El expediente " +
form.getExpediente() + " ya está registrado");
        }
        boolean cambiaPerito = !caso.getPerito().getId().equals(form.getPeritoId());
        copiar(form, caso, cambiaPerito);
        return repo.save(caso);
    }

    /** Regla: un caso con dictamen entregado se conserva por trazabilidad y no se elimina.
*/
    public void eliminar(Long id) {
        Caso caso = obtener(id);
        if (caso.isCerrado()) {
            throw new ReglaNegocioException("El caso " + caso.getExpediente()
                + " tiene el dictamen entregado y se conserva por trazabilidad.");
        }
        repo.delete(caso);
    }
```

5.4 Manejo de errores

Se definieron dos excepciones propias: `ReglaNegocioException` (indica el campo afectado) y `RecursoNoEncontradoException`. En las vistas, el controlador captura la regla de negocio y la muestra junto al campo del formulario con `BindingResult.rejectValue`; si el registro no existe, `WebExceptionHandler` muestra la página de error con código 404. En el API, `ApiExceptionHandler` responde en JSON:

Tabla 6. *Manejo de errores.*

Situación	Vista web	API REST
Datos no válidos	Se vuelve al formulario con mensajes por campo	400 Bad Request + mapa "campos"
Regla de negocio violada	Mensaje en el campo o alerta roja	409 Conflict + "error" y "campo"
Id inexistente	Página de error 404	404 Not Found
Restricción de BD (FK, UNIQUE)	Página de error 409	409 Conflict

6. Integración frontend y backend

La aplicación sigue una arquitectura en capas. El navegador se comunica con los controladores web, que devuelven HTML generado con Thymeleaf; un cliente externo (por ejemplo Postman) usa el API REST, que devuelve JSON. Ambos caminos llegan a los mismos servicios, por lo que

las validaciones y reglas de negocio se aplican igual. Los servicios usan los repositorios JPA, Hibernate traduce las operaciones a SQL y el driver JDBC las ejecuta en MySQL.

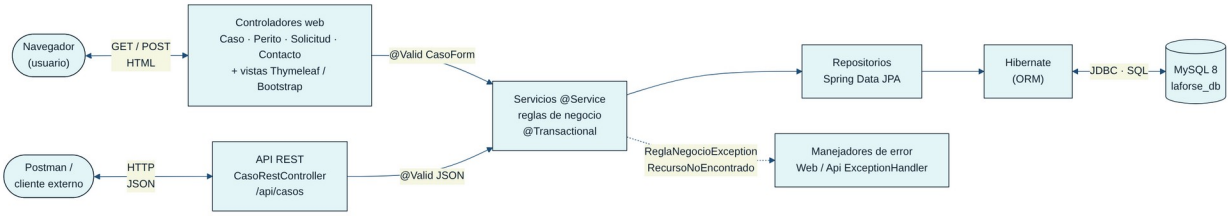


Figura 2. *Arquitectura y flujo de comunicación.*

6.1 Flujo de una operación (registrar un caso)

1. El usuario abre `GET /casos/nuevo`. `CasoController` arma un `CasoForm` con el siguiente expediente sugerido y envía al `Model` las listas de peritos activos, servicios y estados.
2. Thymeleaf enlaza el formulario con `th:object="${caso}"` y `th:field`, y Bootstrap aplica la validación del navegador.
3. Al enviar, `POST /casos` llega al controlador; Spring convierte los parámetros en un `CasoForm` y `@Valid` ejecuta Bean Validation. Si hay errores, se vuelve a mostrar el formulario.
4. Si los datos son válidos, `CasoService.registrar()` verifica las reglas, busca el perito y el servicio, y llama a `CasoRepository.save()`. Hibernate ejecuta el `INSERT` en MySQL.
5. El controlador aplica el patrón POST-Redirect-GET: redirige a `/casos` con un mensaje flash y el listado se vuelve a leer desde la base de datos.

6.2 Rutas

Tabla 7. *Rutas del módulo de gestión y del API.*

Método y ruta	Controlador	Respuesta
GET /casos?estado=&q=	CasoController.listar	Vista casos (listado, filtros y conteos)
GET /casos/nuevo[?solicitud=id]	CasoController.nuevo	Vista caso-form vacía o con datos de la solicitud
POST /casos	CasoController.registrar	Redirect a /casos o formulario con errores
GET /casos/{id}/editar · POST /casos/{id}	CasoController.editar / actualizar	Vista caso-form / redirect
POST /casos/{id}/eliminar	CasoController.eliminar	Redirect con alerta de éxito o error
GET /peritos · /peritos/nuevo · /peritos/{id}/editar	PeritoController	Vistas peritos y perito-form
POST /peritos · POST	PeritoController.guardar /	Redirect con alerta

Método y ruta	Controlador	Respuesta
/peritos/{id}/eliminar	eliminar	
GET /solicitudes · POST /solicitudes/{id}/estado · /eliminar	SolicitudController	Vista solicitudes / redirect
GET/POST /contacto	ContactoController	Formulario público; guarda en la tabla solicitud
GET, POST /api/casos · GET, PUT, DELETE /api/casos/{id}	CasoRestController	JSON (200, 201, 204, 400, 404, 409)

7. Interfaces del sistema

Las capturas se tomaron del sistema en ejecución en la PC de desarrollo, conectado a la base de datos MySQL `laforse_db`. Las páginas comparten la barra de navegación (con el nuevo menú «Gestión») y el pie de página del Avance 2; en las figuras se recortó la cabecera para mostrar el contenido.

7.1 Listado

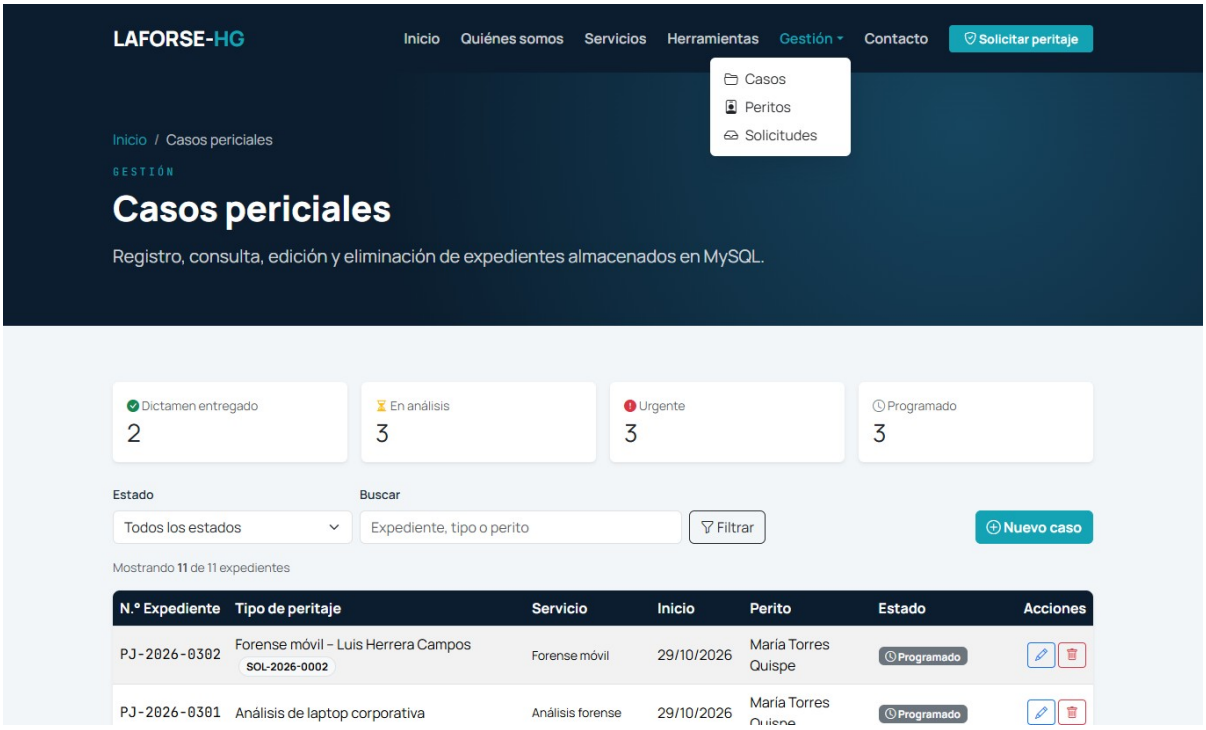


Figura 3. *Menú «Gestión» en la barra de navegación.*

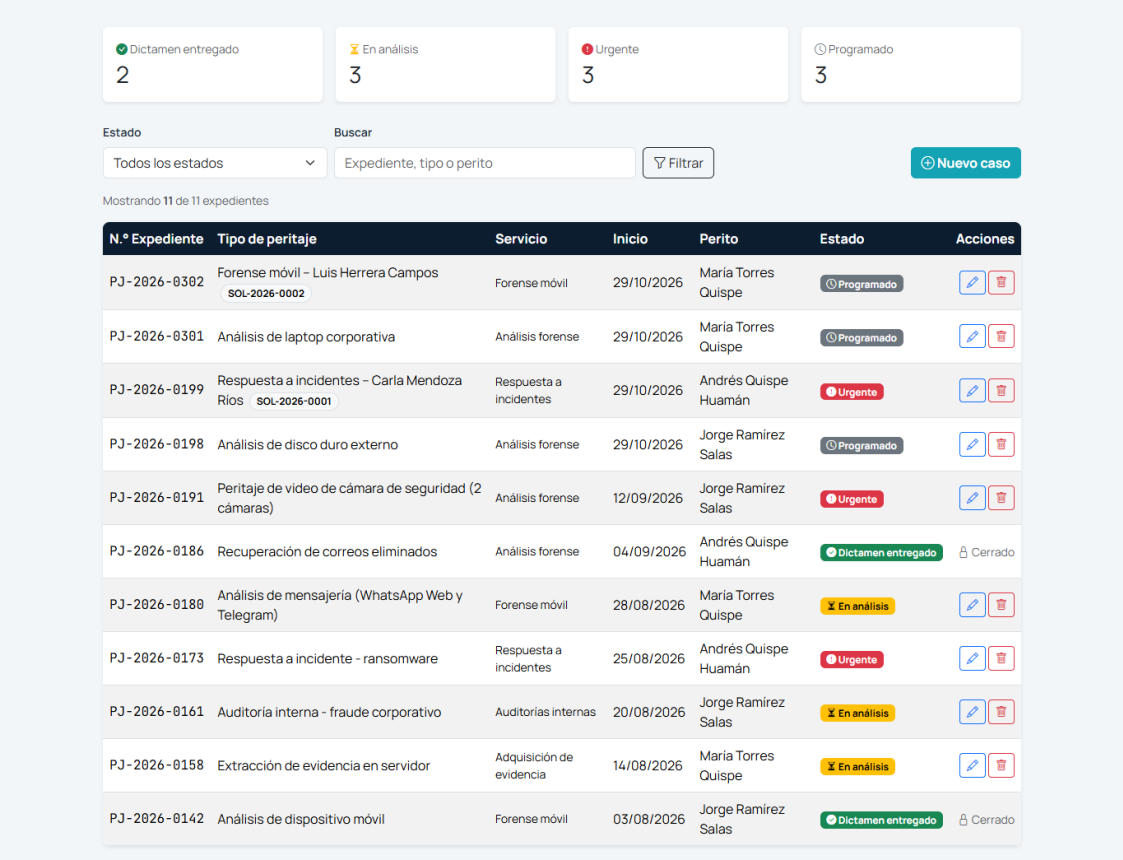


Figura 4. *Listado de casos leído desde MySQL: conteo por estado, filtros, búsqueda y acciones. Los casos cerrados se muestran en solo lectura.*



Figura 5. *Filtro por estado «En análisis» (GET /casos?estado=EN_ANALISIS).*

7.2 Registro

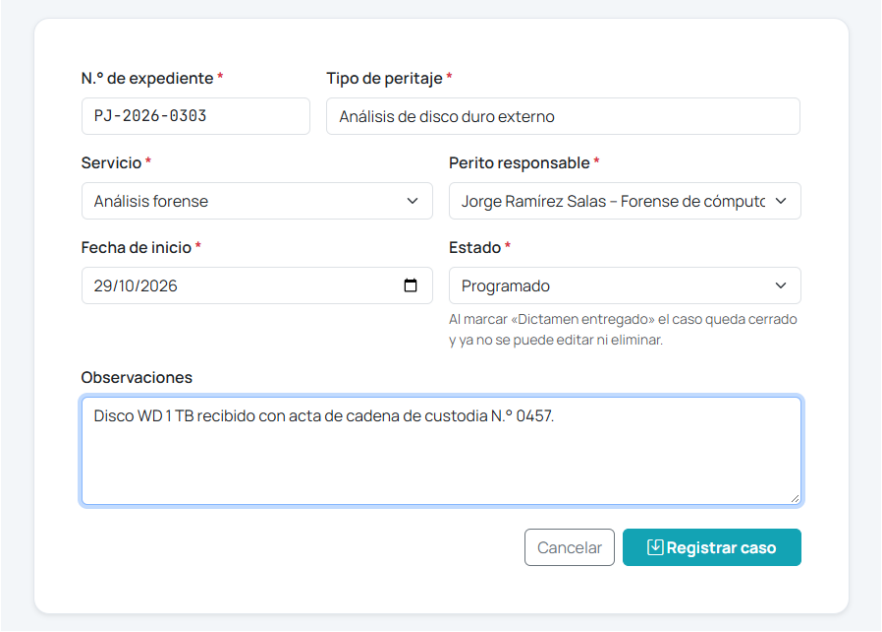


Figura 6. *Formulario de registro de caso con el expediente sugerido por el sistema.*

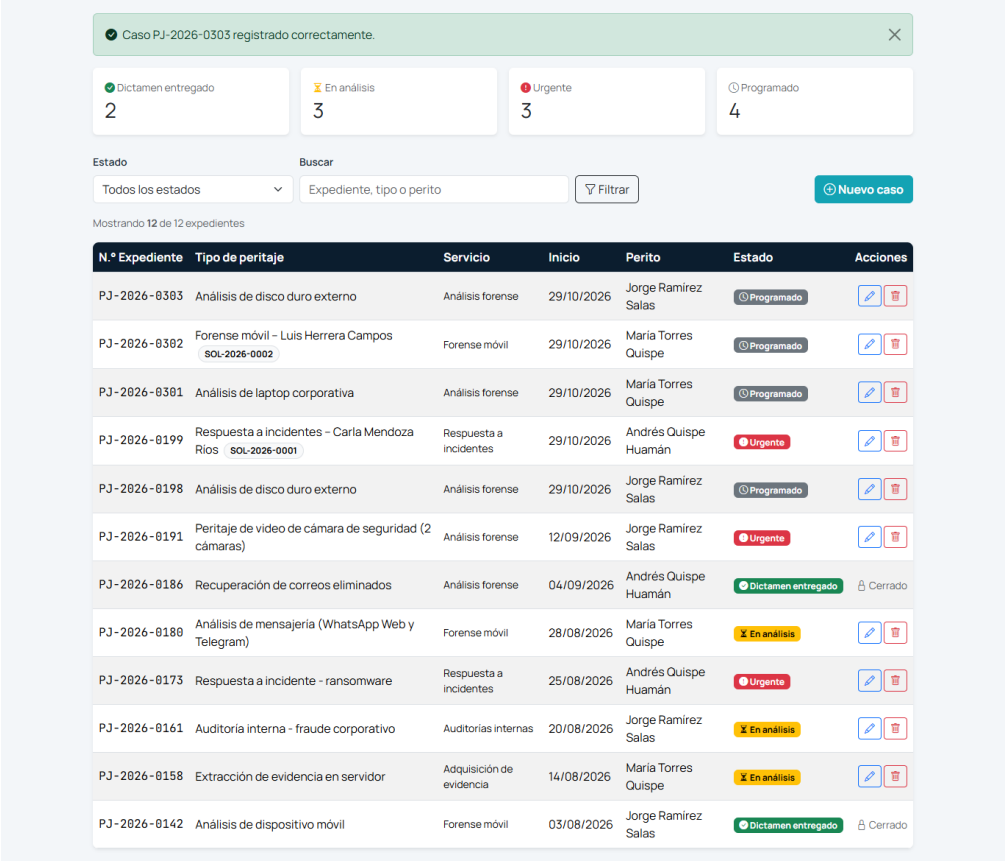


Figura 7. *Caso PJ-2026-0303 registrado en MySQL: alerta de éxito y listado actualizado.*

7.3 Edición

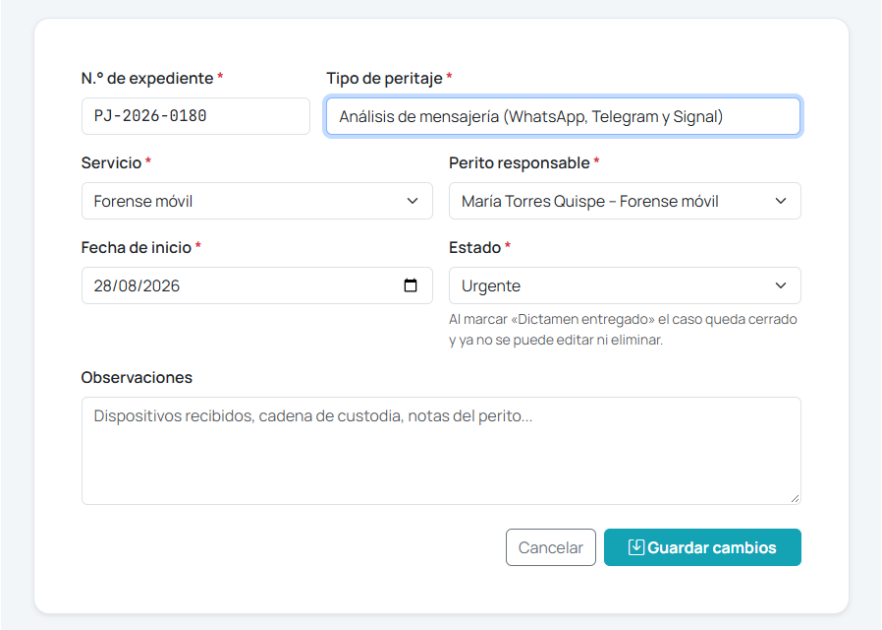


Figura 8. *Edición del caso PJ-2026-0180: cambio de tipo y de estado.*

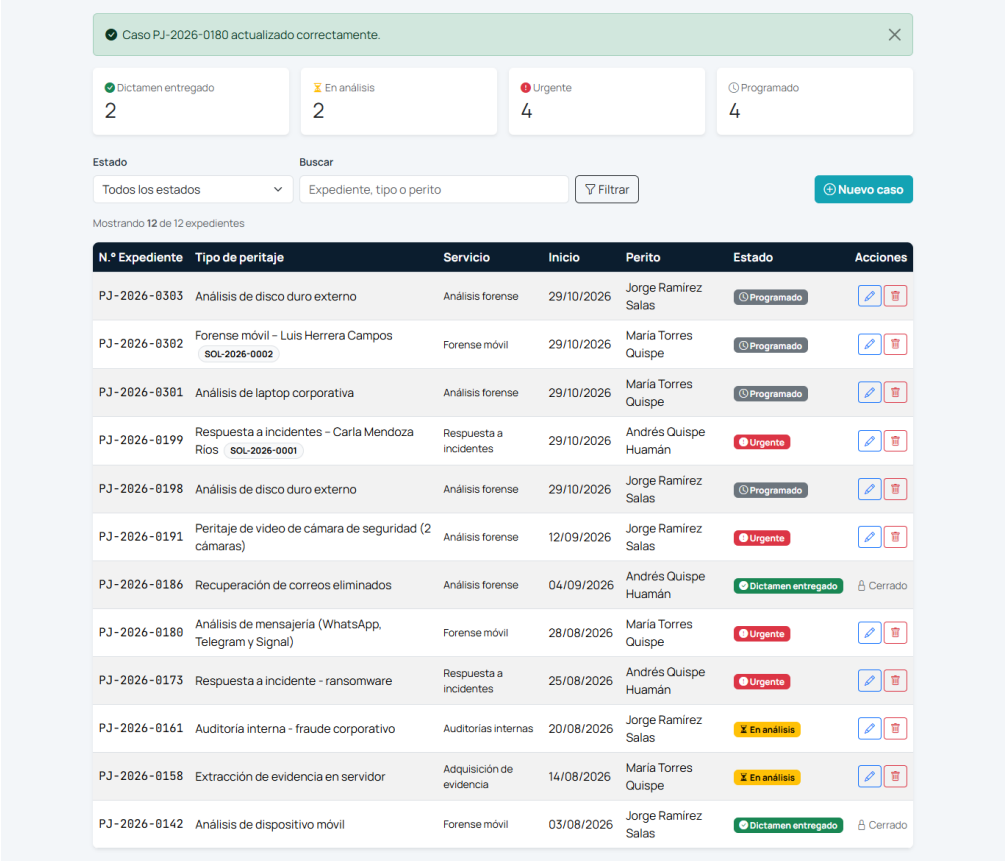


Figura 9. *Caso PJ-2026-0180 actualizado: el estado pasa a «Urgente».*

7.4 Eliminación

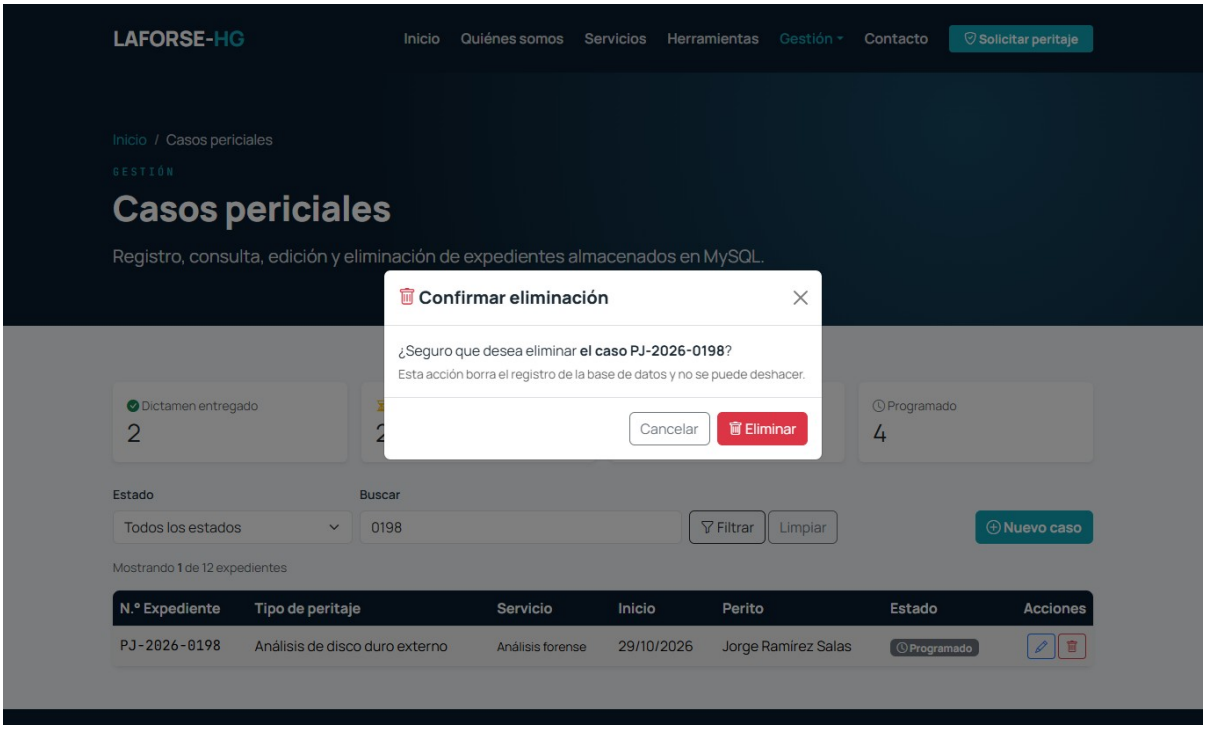


Figura 10. *Modal de confirmación antes de eliminar.*

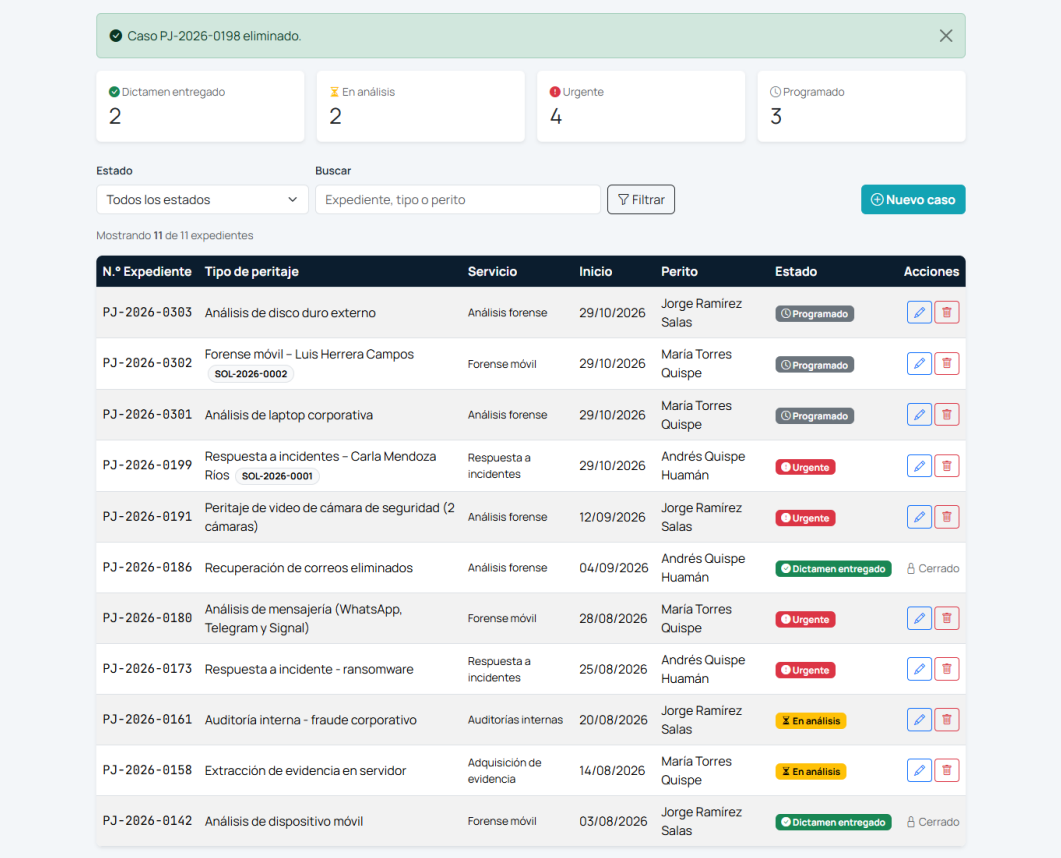


Figura 11. *Caso PJ-2026-0198 eliminado de la base de datos.*

7.5 Validaciones y reglas de negocio

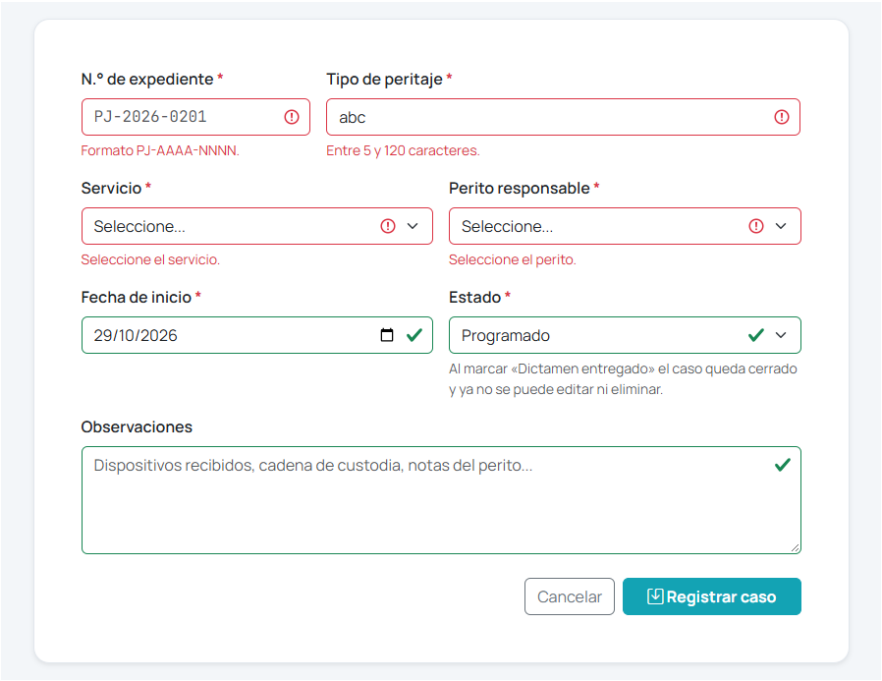


Figura 12. *Validación del navegador (Bootstrap was-validated).*

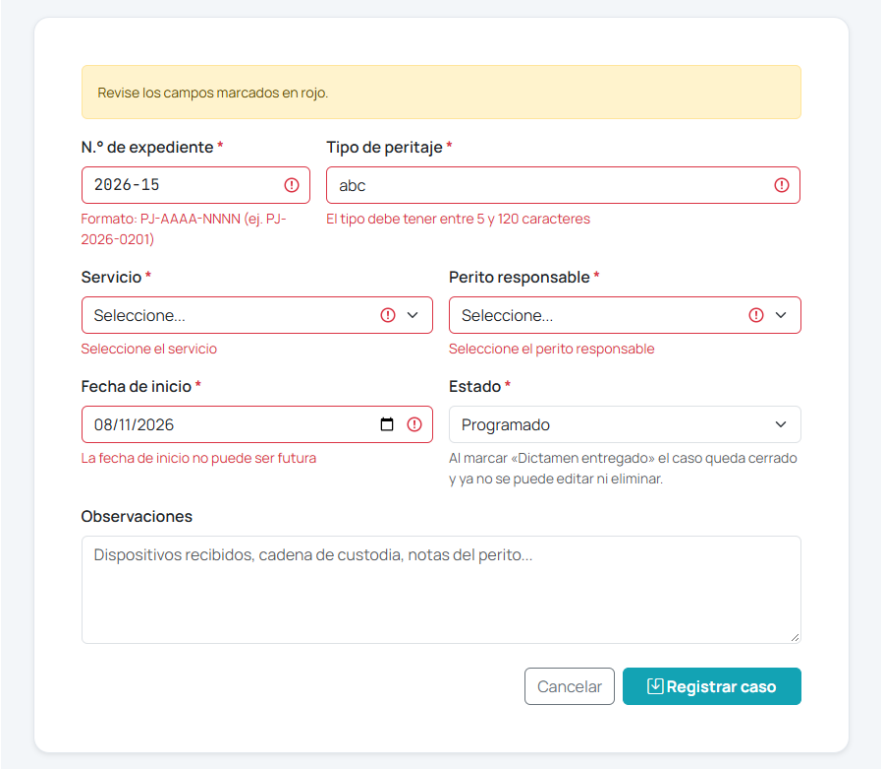


Figura 13. *Validación del servidor (Bean Validation): formato de expediente, longitud, fecha futura y campos obligatorios.*

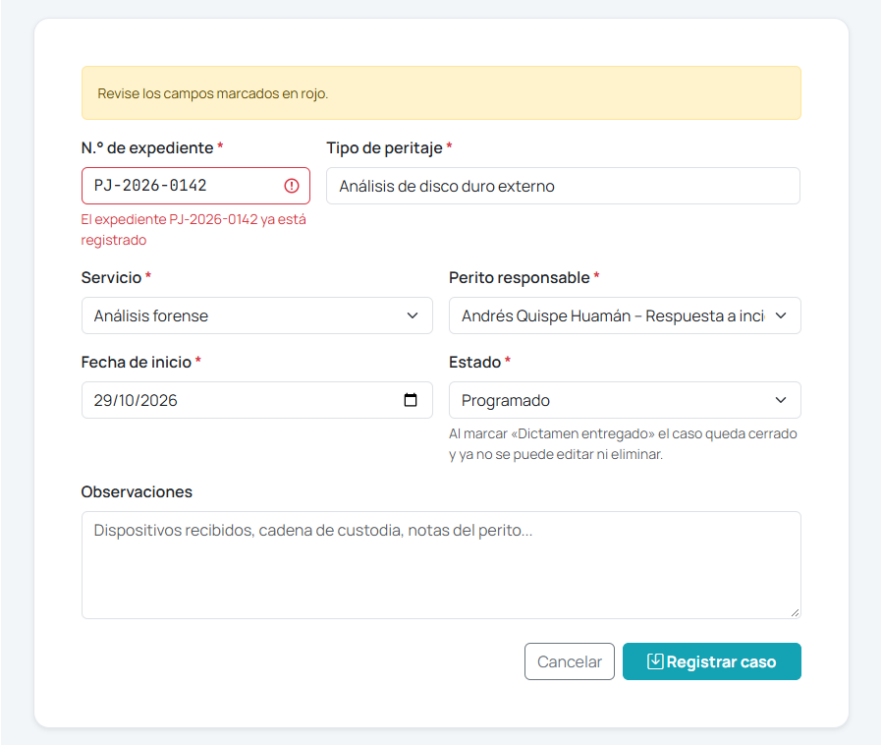


Figura 14. *Regla RN01: expediente duplicado rechazado por el servicio.*



Figura 15. *Regla RN05: no se elimina un perito con casos asignados.*

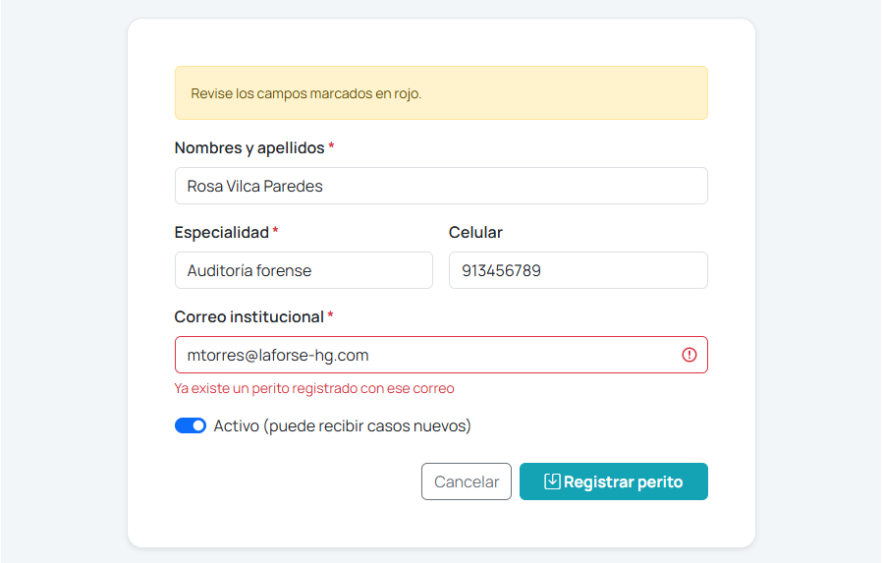


Figura 16. *Regla RN02: correo de perito duplicado.*

7.6 Peritos y solicitudes

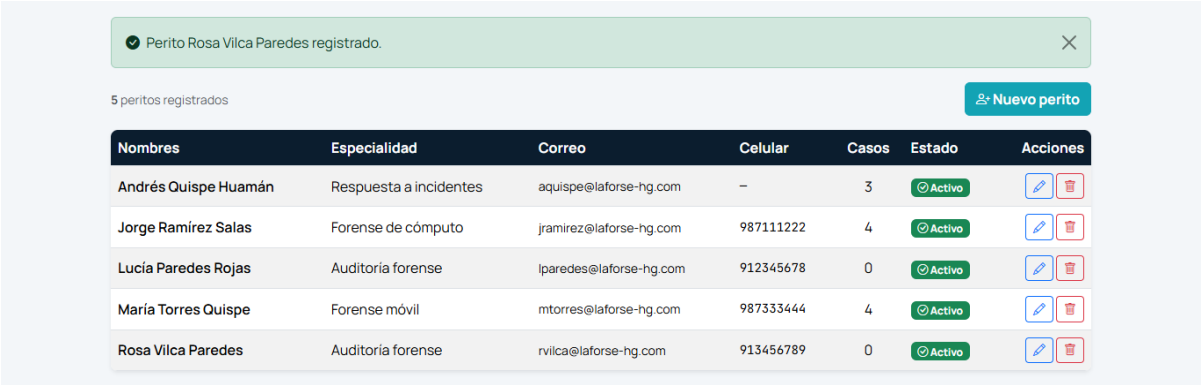


Figura 17. *CRUD de peritos: perito registrado y conteo de casos por perito.*

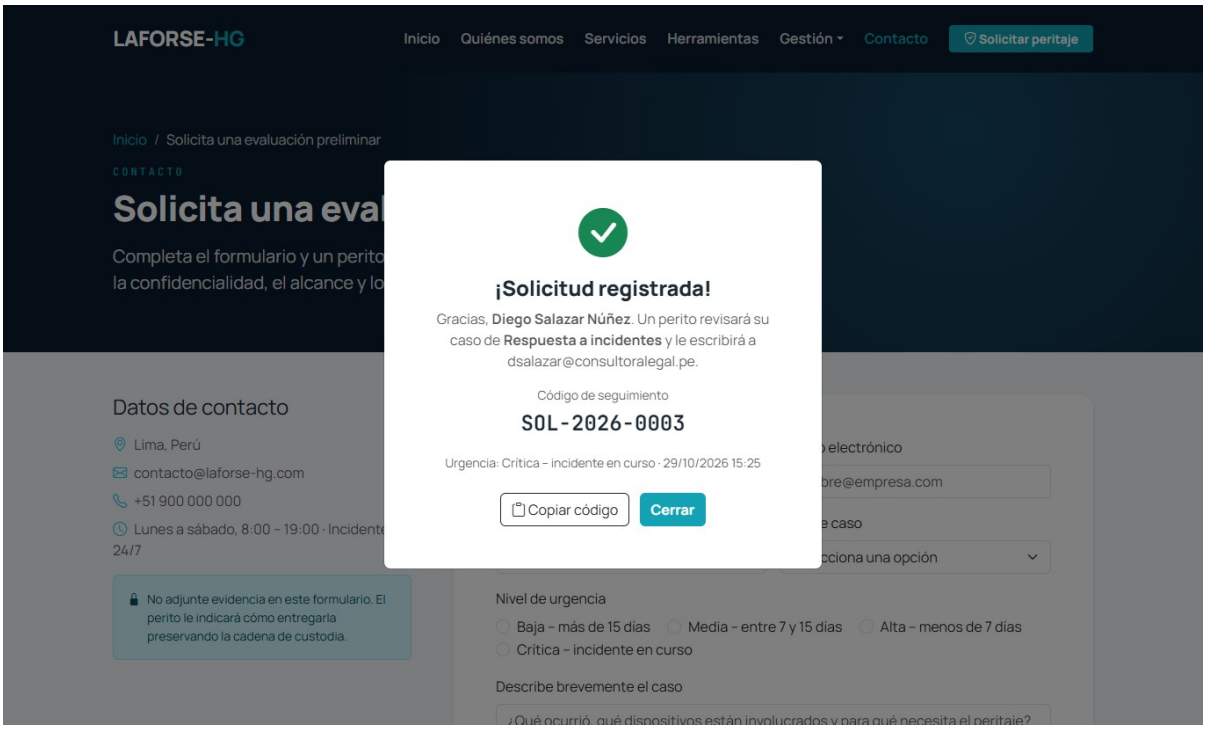


Figura 18. *Formulario público de contacto: la solicitud se guarda en MySQL y se muestra el código de seguimiento.*



Figura 19. *Bandeja de solicitudes con la acción «Abrir caso».*

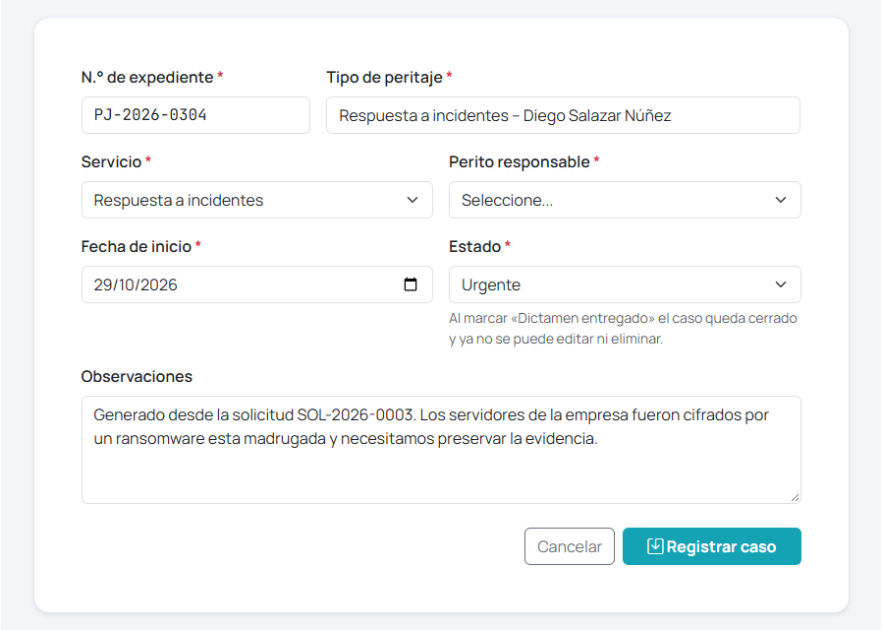


Figura 20. *Caso generado desde la solicitud: servicio, observaciones y estado «Urgente» (RN06) precargados.*



Figura 21. *La solicitud pasa a «Atendida» al registrar el caso.*

7.7 API REST y vista móvil

GET http://localhost:8080/api/casos?estado=URGENTE — 200 OK · application/json


```
[
  {
    "id": 15,
    "expediente": "PJ-2026-0304",
    "tipo": "Respuesta a incidentes – Diego Salazar
Núñez",
    "fechaInicio": "2026-10-29",
    "estado": "URGENTE",
    "peritoId": 3,
    "perito": "Andrés Quispe Huamán",
    "servicioId": 4,
    "servicio": "Respuesta a incidentes",
    "solicitud": "SOL-2026-0003",
    "observaciones": "Generado desde la solicitud SOL-
2026-0003. Los servidores de la empresa fueron cifrados
por un ransomware esta madrugada y necesitamos preservar
la evidencia."
  },
  {
    "id": 10,
    "expediente": "PJ-2026-0199",
    "tipo": "Respuesta a incidentes – Carla Mendoza
Ríos",
    "fechaInicio": "2026-10-29",
    "estado": "URGENTE",
    "peritoId": 3,
    "perito": "Andrés Quispe Huamán",
    "servicioId": 4,
    "servicio": "Respuesta a incidentes",
    "solicitud": "SOL-2026-0001",
    "observaciones": "Generado desde la solicitud SOL-
2026-0001. Los servidores de la empresa fueron cifrados
por un ransomware esta madrugada y necesitamos preservar
la evidencia."
  },
  {
    "id": 7,
    "expediente": "PJ-2026-0191",
    "tipo": "Peritaje de video de cámara de seguridad (2
cámaras)",
    "fechaInicio": "2026-09-12",
    "estado": "URGENTE",
    "peritoId": 1,
    "perito": "Jorge Ramírez Salas",
    "servicioId": 2,
    "servicio": "Análisis forense",
    "solicitud": null,
    "observaciones": null
  },
  {
    "id": 5,
    "expediente": "PJ-2026-0180",
    "tipo": "Análisis de mensajería (WhatsApp, Telegram
y Signal)",
    "fechaInicio": "2026-08-28",
    "estado": "URGENTE",
    "peritoId": 2,
    "perito": "María Torres Quispe",
    "servicioId": 5,
    "servicio": "Forense móvil",
    "solicitud": null,
    "observaciones": null
  },
  {
    "id": 4,
    "expediente": "PJ-2026-0173",
    "tipo": "Respuesta a incidente - ransomware",
    "fechaInicio": "2026-08-25",
    "estado": "URGENTE",
    "peritoId": 3,
    "perito": "Andrés Quispe Huamán",
    "servicioId": 4,
    "servicio": "Respuesta a incidentes",
    "solicitud": null,
    "observaciones": null
  }
]
```


Figura 22. *Respuesta JSON real de GET /api/casos?estado=URGENTE (casos leídos desde MySQL).*

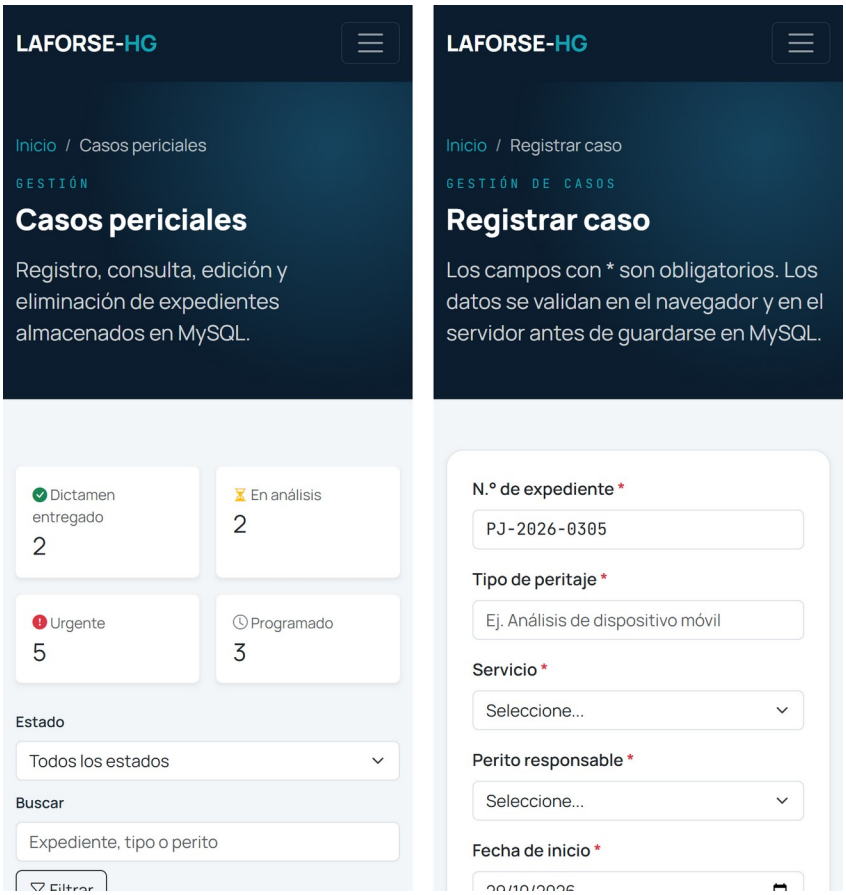


Figura 23. *Vista móvil (390 px): listado y formulario de casos.*

8. Pruebas realizadas

8.1 Pruebas funcionales

Tabla 8. *Casos de prueba funcional.*

ID	Caso de prueba	Resultado esperado	Evidencia
PF01	Registrar caso con datos válidos	Se guarda en la tabla caso y aparece en el listado	Fig. listado + SELECT
PF02	Registrar caso con campos vacíos o inválidos	No se guarda; mensajes por campo	Validaciones
PF03	Registrar expediente existente (PJ-2026-0142)	Error «ya está registrado» (RN01)	Expediente duplicado
PF04	Fecha de inicio futura	Error «no puede ser futura»	Validación servidor
PF05	Editar caso abierto	Se actualiza el registro en MySQL	Edición
PF06	Editar o eliminar caso con	Acción bloqueada (RN04)	Listado (candado)

ID	Caso de prueba	Resultado esperado	Evidencia
	dictamen entregado		
PF07	Eliminar caso abierto	Modal de confirmación y registro borrado	Eliminación
PF08	Eliminar perito con casos	Mensaje de error, el perito se conserva (RN05)	Regla perito
PF09	Registrar perito con correo repetido	Error en el campo correo (RN02)	Correo duplicado
PF10	Enviar solicitud desde /contacto	Se guarda en solicitud con código SOL-AAAA-NNNN	Confirmación
PF11	Abrir caso desde solicitud crítica	Caso «Urgente» y solicitud «Atendida» (RN06)	Solicitudes
PF12	CRUD por API REST (curl / Postman)	201, 200, 204 y 404 tras eliminar	API

8.2 Almacenamiento en MySQL

Después de las pruebas funcionales (registrar PJ-2026-0198, editar PJ-2026-0180, eliminar PJ-2026-0197 y convertir la solicitud SOL-2026-0001 en el caso PJ-2026-0199) se consultó MySQL 8.4 con el cliente `mysql` para verificar que los datos quedaron guardados y que las claves foráneas relacionan correctamente las tablas:

Consultas de verificación

```
SELECT c.expediente, c.tipo, c.estado, p.nombres AS perito, s.nombre AS servicio
  FROM caso c
  JOIN perito   p ON p.id = c.perito_id
  JOIN servicio s ON s.id = c.servicio_id
 ORDER BY c.fecha_inicio DESC;

SELECT codigo, nombre, urgencia, estado FROM solicitud ORDER BY fecha DESC;
```

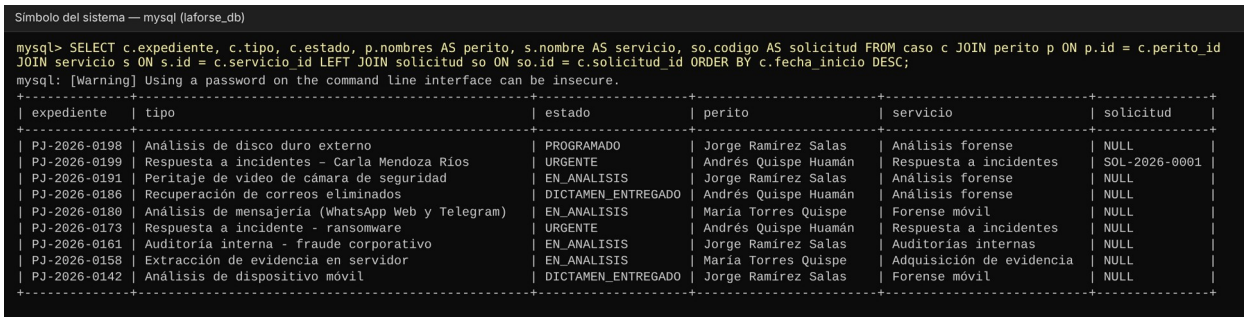


Figura 24. *MySQL: tabla caso después de las pruebas. Aparecen PJ-2026-0198 (registrado), PJ-2026-0180 editado y PJ-2026-0199 vinculado a SOL-2026-0001; PJ-2026-0197 ya no existe.*

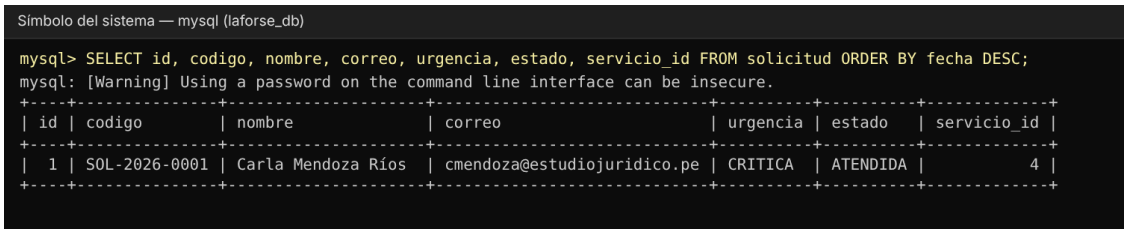


Figura 25. *MySQL: la solicitud enviada desde el formulario de contacto quedó guardada y en estado ATENDIDA (RN06).*

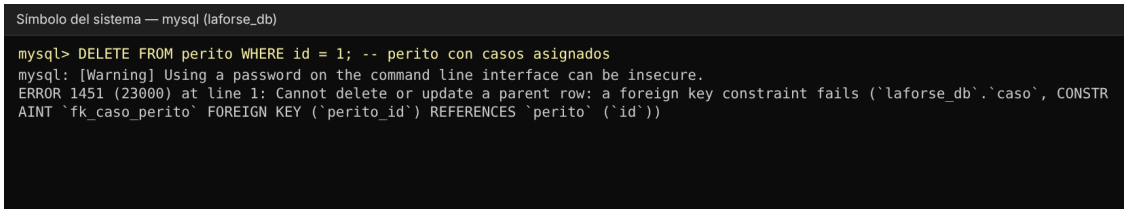


Figura 26. *MySQL: la clave foránea fk_caso_perito impide borrar un perito con casos (integridad referencial).*

8.3 Pruebas del API REST

El API se probó con `curl` contra la aplicación conectada a MySQL. El proyecto también incluye la colección `docs/LAFORSE-API.postman_collection.json` con ocho peticiones: listar, filtrar, registrar, obtener, actualizar, eliminar, error de validación (400) y expediente duplicado (409).

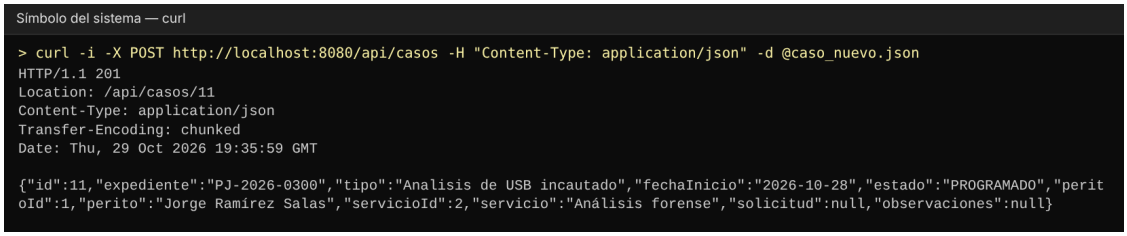


Figura 27. *API REST: POST /api/casos registra el caso PJ-2026-0300 y responde 201 Created con su ubicación.*



Figura 28. *API REST: un expediente duplicado responde 409 Conflict con el mensaje de la regla RN01.*

8.4 Pruebas automatizadas

Se ejecutan con `mvn test`. Usan una base de datos H2 en memoria en modo MySQL (`src/test/resources/application.properties`), por lo que no modifican los datos reales.

Tabla 9. *Pruebas automatizadas (JUnit 5 + MockMvc).*

Clase de prueba	N.°	Qué verifica
WebControllerTest	5	Todas las páginas responden 200; filtro de casos; 404 de servicio inexistente; formulario de contacto inválido y válido (guarda en BD)

Clase de prueba	N.°	Qué verifica
CasoCrudTest	11	Expediente sugerido; registrar; datos inválidos; expediente duplicado; editar; caso cerrado bloqueado; eliminar; 404; reglas de peritos (eliminar, registrar, correo duplicado)
CasoApiTest	4	Listado JSON; CRUD completo por API (201, 200, 204, 404); validación 400; regla de negocio 409

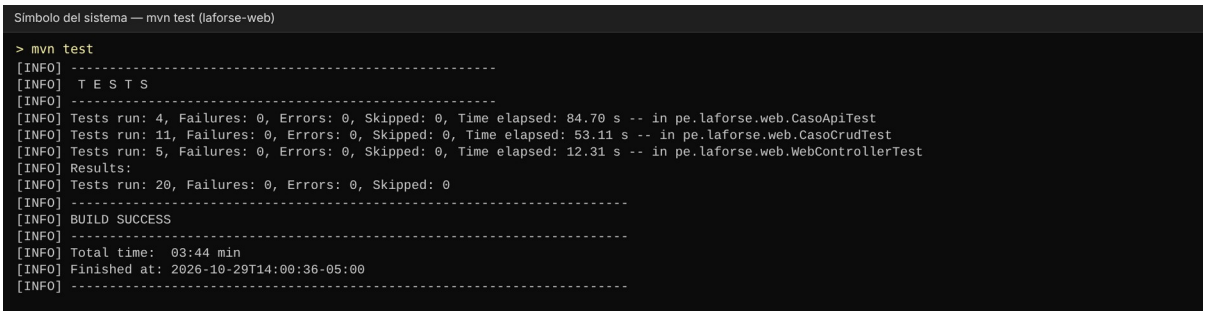


Figura 29. *Resultado de mvn test: 20 pruebas ejecutadas, 0 fallos, BUILD SUCCESS.*

9. Trabajo pendiente

- **Autenticación y roles** con Spring Security: hoy el módulo «Gestión» es accesible sin iniciar sesión; se separarán los roles administrador y perito.
- **Historial de cambios del caso** (bitácora de estados con fecha y usuario) para reforzar la cadena de custodia.
- **Carga de documentos** del expediente (actas, informes) asociados a cada caso.
- **Notificaciones por correo** al cliente cuando su solicitud es atendida.
- **Paginación y exportación** del listado de casos a PDF o Excel.
- **Despliegue** en un servidor con MySQL remoto y variables de entorno para las credenciales.

10. Conclusiones

1. Spring Data JPA con Hibernate permitió persistir los datos en MySQL sin escribir SQL para las operaciones comunes: bastó con anotar las entidades y declarar interfaces de repositorio. Las consultas derivadas y una consulta JPQL cubrieron los filtros y las validaciones de unicidad.
2. Separar controladores, servicios y repositorios hizo posible reutilizar la misma lógica para las vistas Thymeleaf y para el API REST: las reglas de negocio se escriben una sola vez y se aplican igual desde el navegador o desde Postman.
3. La validación en dos niveles (navegador y servidor), junto con las restricciones de la base de datos (UNIQUE, FOREIGN KEY y CHECK), da tres barreras ante datos incorrectos y mejora la experiencia del usuario con mensajes claros por campo.

4. Las pruebas automatizadas con MockMvc y una base en memoria permitieron comprobar el CRUD y las reglas de negocio en segundos después de cada cambio, lo que redujo errores durante la integración.

11. Anexos

Anexo A. Especificación de casos de uso

Tabla 10. *Lista de casos de uso.*

ID	Caso de uso	Actor
UC01	Consultar servicios	Cliente
UC02	Enviar solicitud de peritaje	Cliente
UC03	Validar datos	(incluido)
UC04	Registrar caso	Administrador
UC05	Listar y filtrar casos	Administrador
UC06	Editar caso	Administrador
UC07	Eliminar caso	Administrador
UC08	Gestionar peritos (CRUD)	Administrador
UC09	Gestionar solicitudes	Administrador
UC10	Abrir caso desde solicitud	Administrador (extiende UC09)
UC11	Consumir API REST de casos	Cliente API

Tabla 11. *Especificación de UC04 – Registrar caso.*

UC04 – Registrar caso	
Actor	Administrador
Descripción	Registra un nuevo expediente pericial y lo asigna a un perito.
Precondición	El administrador está en el módulo Gestión. Existen servicios y al menos un perito activo.
Flujo básico	1. El administrador pulsa «Nuevo caso». 2. El sistema muestra el formulario con el siguiente expediente sugerido, la fecha de hoy y el estado «Programado». 3. El administrador ingresa el tipo, selecciona servicio y perito, y opcionalmente escribe observaciones. 4. El administrador pulsa «Registrar caso». 5. El sistema valida los datos (UC03) y las reglas RN01 y RN03. 6. El sistema guarda el caso en MySQL y muestra el listado con el mensaje «Caso … registrado correctamente».
Flujos alternos	5a. Datos no válidos: el sistema muestra los errores junto a cada campo y no guarda. 5b. Expediente duplicado o perito inactivo: el sistema muestra el

UC04 – Registrar caso	
	mensaje en el campo correspondiente.
Postcondición	El caso queda registrado en la tabla caso con sus claves foráneas a perito y servicio.

Tabla 12. *Especificación de UC05 – Listar y filtrar casos.*

UC05 – Listar y filtrar casos	
Actor	Administrador
Descripción	Consulta los expedientes registrados.
Precondición	Ninguna.
Flujo básico	1. El administrador entra a Gestión › Casos. 2. El sistema muestra el conteo por estado y la tabla de casos ordenada por fecha de inicio. 3. El administrador selecciona un estado y/o escribe un texto y pulsa «Filtrar». 4. El sistema muestra solo los casos que coinciden y el total mostrado.
Flujos alternos	4a. Sin resultados: el sistema muestra «No hay expedientes que coincidan con el filtro».
Postcondición	Ninguna (solo lectura).

Tabla 13. *Especificación de UC06 – Editar caso.*

UC06 – Editar caso	
Actor	Administrador
Descripción	Modifica los datos o el estado de un caso abierto.
Precondición	El caso existe y no tiene el dictamen entregado.
Flujo básico	1. El administrador pulsa el botón Editar del caso. 2. El sistema muestra el formulario con los datos actuales. 3. El administrador modifica los campos y pulsa «Guardar cambios». 4. El sistema valida los datos (UC03) y la regla RN01. 5. El sistema actualiza el registro y muestra el listado con un mensaje de éxito.
Flujos alternos	1a. Caso cerrado (RN04): el sistema no muestra el botón; si se accede por URL, redirige con un mensaje de error. 4a. Datos no válidos o expediente duplicado: el sistema muestra los errores. 1b. Id inexistente: el sistema muestra la página de error 404.

UC06 – Editar caso	
Postcondición	El registro queda actualizado en MySQL.

Tabla 14. *Especificación de UC07 – Eliminar caso.*

UC07 – Eliminar caso	
Actor	Administrador
Descripción	Elimina un caso registrado por error.
Precondición	El caso existe y no tiene el dictamen entregado.
Flujo básico	1. El administrador pulsa el botón Eliminar. 2. El sistema muestra el modal de confirmación con el número de expediente. 3. El administrador confirma. 4. El sistema elimina el registro y muestra el mensaje «Caso … eliminado».
Flujos alternos	3a. El administrador cancela: no se realiza ningún cambio. 4a. Caso cerrado (RN04): el sistema muestra un mensaje de error y conserva el caso.
Postcondición	El registro ya no existe en la tabla caso.

Tabla 15. *Especificación de UC02 – Enviar solicitud de peritaje.*

UC02 – Enviar solicitud de peritaje	
Actor	Cliente
Descripción	El cliente solicita una evaluación desde el sitio público.
Precondición	Ninguna.
Flujo básico	1. El cliente abre Contacto. 2. Ingresa nombre, correo, celular, tipo de caso, urgencia y descripción, y acepta el aviso de privacidad. 3. Pulsa «Enviar solicitud». 4. El sistema valida los datos (UC03). 5. El sistema guarda la solicitud con estado «Pendiente» y muestra el código de seguimiento SOL-AAAA-NNNN.
Flujos alternos	4a. Datos no válidos: el sistema muestra los errores por campo.
Postcondición	La solicitud queda en la tabla solicitud.

Tabla 16. *Especificación de UC10 – Abrir caso desde solicitud.*

UC10 – Abrir caso desde solicitud	
Actor	Administrador
Descripción	Convierte una solicitud pendiente en un caso.
Precondición	La solicitud está en estado «Pendiente».
Flujo básico	1. En Gestión › Solicitudes, el administrador pulsa «Abrir caso». 2. El sistema muestra el formulario de caso con servicio, tipo y observaciones tomados de la solicitud; si la urgencia es crítica, el estado es «Urgente». 3. El administrador selecciona el perito y registra el caso (UC04). 4. El sistema guarda el caso con la referencia a la solicitud y cambia la solicitud a «Atendida».
Flujos alternos	3a. Igual que los flujos alternos de UC04.
Postcondición	Caso creado y solicitud atendida.

Tabla 17. *Especificación de UC08 – Gestionar peritos (CRUD).*

UC08 – Gestionar peritos (CRUD)	
Actor	Administrador
Descripción	Registra, lista, edita, desactiva y elimina peritos.
Precondición	Ninguna.
Flujo básico	1. El administrador entra a Gestión › Peritos. 2. El sistema lista los peritos con su número de casos y estado. 3. El administrador registra o edita un perito y guarda. 4. El sistema valida los datos y la regla RN02 y guarda el registro.
Flujos alternos	4a. Correo duplicado: error en el campo correo. Eliminar perito con casos (RN05): el sistema muestra un error y sugiere desactivarlo.
Postcondición	El perito queda registrado, actualizado o eliminado.

Anexo B. Diagrama de casos de uso

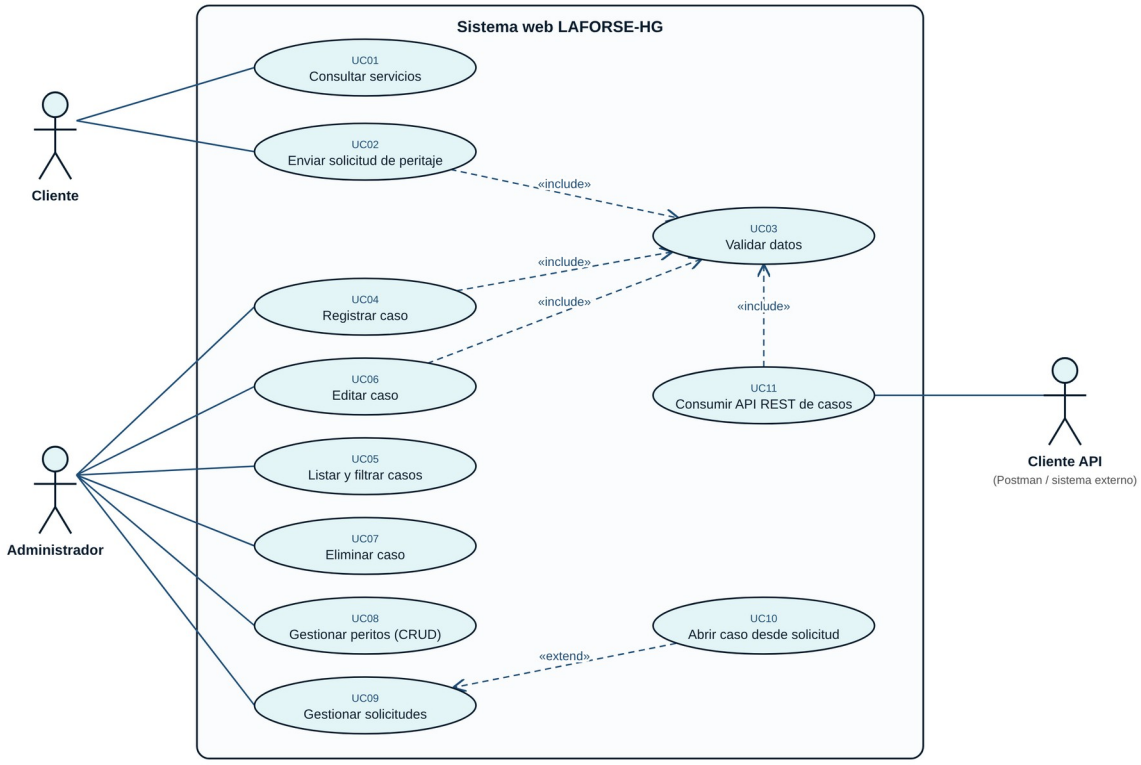


Figura 30. *Diagrama de casos de uso del sistema.*

Anexo C. Diagrama de secuencia: Registrar caso

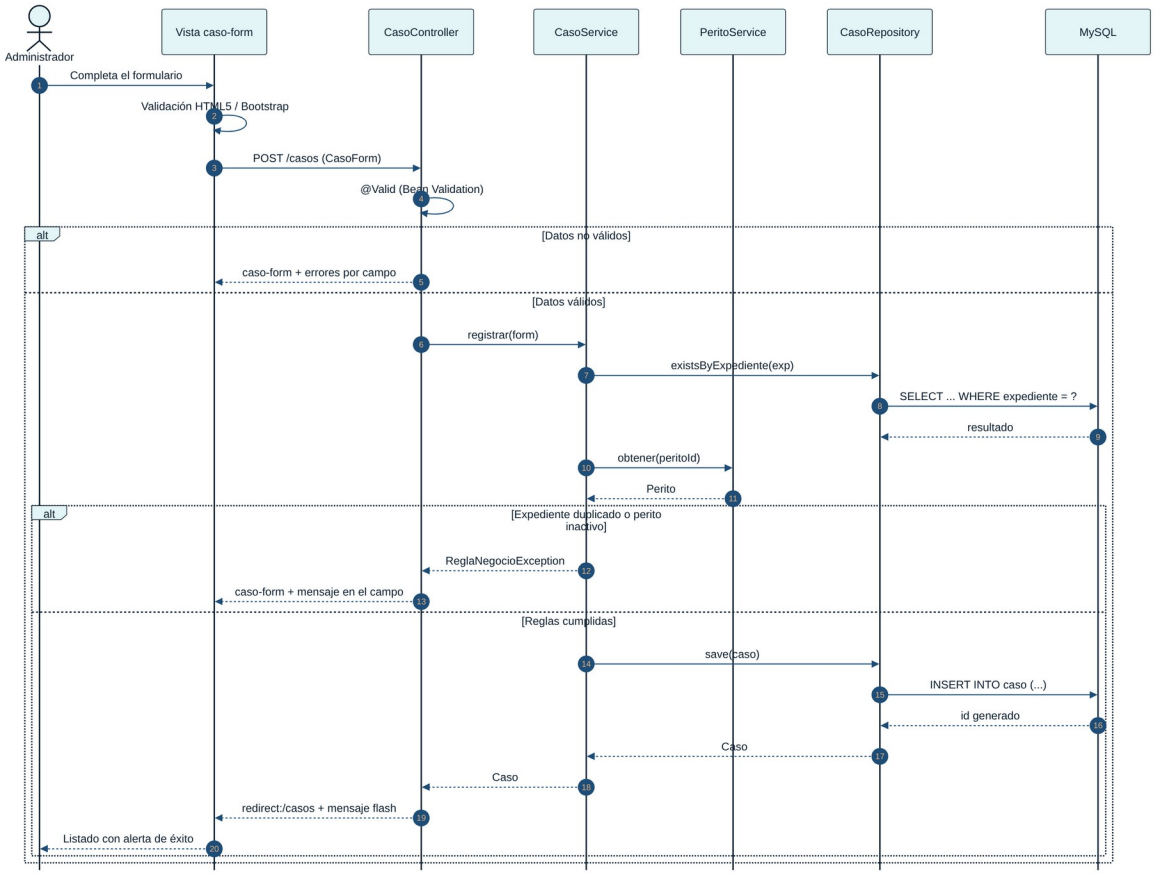


Figura 31. *Diagrama de secuencia de la funcionalidad principal «Registrar caso».*

Anexo D. Diagrama de clases

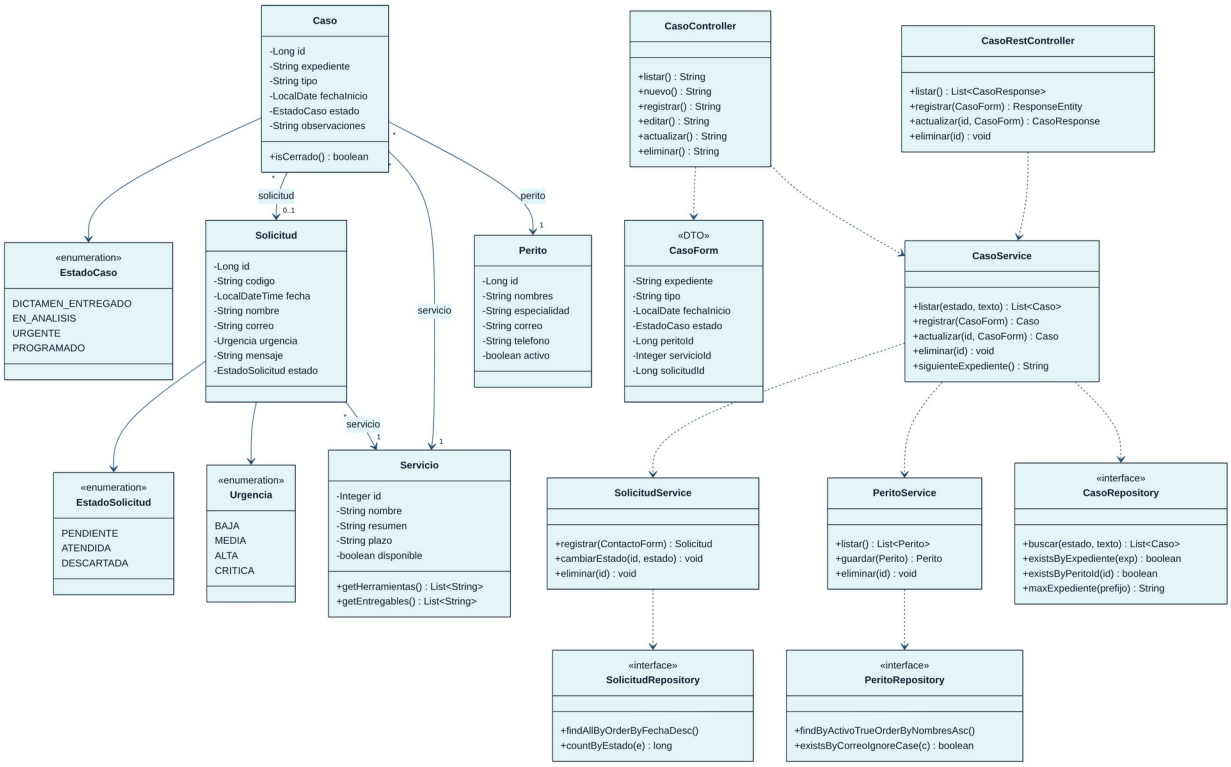


Figura 32. *Diagrama de clases del sistema (entidades, DTO, repositorios, servicios y controladores).*

Anexo E. Código fuente relevante

model/Caso.java (entidad JPA, fragmento)

```
/** Expediente pericial (tabla caso). Se relaciona con perito, servicio y, opcionalmente, con
la solicitud de origen. */
@Entity
@Table(name = "caso")
public class Caso {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 12)
    private String expediente;

    @Column(nullable = false, length = 120)
    private String tipo;

    @Column(name = "fecha_inicio", nullable = false)
    private LocalDate fechaInicio;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
```

```
    private EstadoCaso estado;

    @Column(length = 500)
    private String observaciones;

    @ManyToOne(optional = false)
    @JoinColumn(name = "perito_id", nullable = false)
    private Perito perito;

    @ManyToOne(optional = false)
    @JoinColumn(name = "servicio_id", nullable = false)
    private Servicio servicio;

    @ManyToOne
    @JoinColumn(name = "solicitud_id")
    private Solicitud solicitud;
```

repository/CasoRepository.java

```
public interface CasoRepository extends JpaRepository<Caso, Long> {

    /** Búsqueda combinada: estado opcional y texto opcional en expediente, tipo o perito. */
    @Query("""
           SELECT c FROM Caso c
           WHERE (:estado IS NULL OR c.estado = :estado)
             AND (:texto IS NULL
                  OR LOWER(c.expediente) LIKE LOWER(CONCAT('%', :texto, '%'))
                  OR LOWER(c.tipo) LIKE LOWER(CONCAT('%', :texto, '%'))
                  OR LOWER(c.perito.nombres) LIKE LOWER(CONCAT('%', :texto, '%')))
           ORDER BY c.fechaInicio DESC, c.id DESC
           """)
    List<Caso> buscar(@Param("estado") EstadoCaso estado, @Param("texto") String texto);

    long countByEstado(EstadoCaso estado);
    boolean existsByExpediente(String expediente);
    boolean existsByExpedienteAndIdNot(String expediente, Long id);
    boolean existsByPeritoId(Long peritoId);
    long countByPeritoId(Long peritoId);
    boolean existsBySolicitudId(Long solicitudId);

    /** Mayor número de expediente del año (los números tienen ancho fijo, por eso MAX
funciona). */
    @Query("SELECT MAX(c.expediente) FROM Caso c WHERE c.expediente LIKE CONCAT(:prefijo,
'%')")
    String maxExpediente(@Param("prefijo") String prefijo);
}
```

controller/CasoController.java (registrar, editar y eliminar)

```
    // ---------- CREATE ----------
    @GetMapping("/nuevo")
```

```
    public String nuevo(@RequestParam(required = false) Long solicitud, Model model) {
        model.addAttribute("caso", casoService.nuevoFormulario(solicitud));
        return formulario(model, null);
    }

    @PostMapping
    public String registrar(@Valid @ModelAttribute("caso") CasoForm caso, BindingResult
result,
                            Model model, RedirectAttributes flash) {
        if (result.hasErrors()) {
            return formulario(model, null);
        }
        try {
            Caso nuevo = casoService.registrar(caso);
            flash.addFlashAttribute("exito", "Caso " + nuevo.getExpediente() + " registrado
correctamente.");
            return "redirect:/casos";
        } catch (ReglaNegocioException ex) {
            rechazar(result, ex);
            return formulario(model, null);
        }
    }

    // ---------- UPDATE ----------
    @GetMapping("/{id}/editar")
    public String editar(@PathVariable Long id, Model model, RedirectAttributes flash) {
        Caso c = casoService.obtener(id);
        if (c.isCerrado()) {
            flash.addFlashAttribute("error", "El caso " + c.getExpediente() + " tiene el
dictamen entregado y ya no se puede modificar.");
            return "redirect:/casos";
        }
        model.addAttribute("caso", casoService.aFormulario(c));
        return formulario(model, id);
    }

    @PostMapping("/{id}")
    public String actualizar(@PathVariable Long id, @Valid @ModelAttribute("caso") CasoForm
caso,
                             BindingResult result, Model model, RedirectAttributes flash) {
        if (result.hasErrors()) {
            return formulario(model, id);
        }
        try {
            Caso c = casoService.actualizar(id, caso);
            flash.addFlashAttribute("exito", "Caso " + c.getExpediente() + " actualizado
correctamente.");
            return "redirect:/casos";
        } catch (ReglaNegocioException ex) {
            rechazar(result, ex);
            return formulario(model, id);
        }
    }
```

```
    // ---------- DELETE ----------
    @PostMapping("/{id}/eliminar")
    public String eliminar(@PathVariable Long id, RedirectAttributes flash) {
        try {
            String exp = casoService.obtener(id).getExpediente();
            casoService.eliminar(id);
            flash.addFlashAttribute("exito", "Caso " + exp + " eliminado.");
        } catch (ReglaNegocioException ex) {
            flash.addFlashAttribute("error", ex.getMessage());
        }
        return "redirect:/casos";
    }
```

controller/api/CasoRestController.java

```
@RestController
@RequestMapping("/api/casos")
public class CasoRestController {

    private final CasoService casoService;

    public CasoRestController(CasoService casoService) {
        this.casoService = casoService;
    }

    @GetMapping
    public List<CasoResponse> listar(@RequestParam(required = false) EstadoCaso estado,
                                     @RequestParam(required = false) String q) {
        return casoService.listar(estado, q).stream().map(CasoResponse::de).toList();
    }

    @GetMapping("/{id}")
    public CasoResponse obtener(@PathVariable Long id) {
        return CasoResponse.de(casoService.obtener(id));
    }

    @PostMapping
    public ResponseEntity<CasoResponse> registrar(@Valid @RequestBody CasoForm form) {
        Caso c = casoService.registrar(form);
        return ResponseEntity.created(URI.create("/api/casos/" +
c.getId())).body(CasoResponse.de(c));
    }

    @PutMapping("/{id}")
    public CasoResponse actualizar(@PathVariable Long id, @Valid @RequestBody CasoForm form)
{
        return CasoResponse.de(casoService.actualizar(id, form));
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
```

```
    public void eliminar(@PathVariable Long id) {
        casoService.eliminar(id);
    }
}
```

templates/caso-form.html (campo con validación cliente y servidor)

```
                <div class="col-md-4">
                  <label for="expediente" class="form-label fw-semibold req">N.° de
expediente</label>
                  <input type="text" id="expediente" th:field="*{expediente}" class="form-
control font-mono" required
                         pattern="PJ-[0-9]{4}-[0-9]{4}" th:errorclass="is-invalid"
placeholder="PJ-2026-0201">
                  <div class="invalid-feedback" th:if="${#fields.hasErrors('expediente')}"
th:text="${#fields.errors('expediente')[0]}"></div>
                  <div class="invalid-feedback" th:unless="$
{#fields.hasErrors('expediente')}">Formato PJ-AAAA-NNNN.</div>
                </div>
```

Anexo F. Ejecución del proyecto

1. Ejecutar `database/laforse_db.sql` en MySQL (Workbench o cliente `mysql`).
2. Revisar `spring.datasource.username` y `spring.datasource.password` en `application.properties`.
3. Abrir el proyecto en NetBeans y pulsar Run (o `mvn spring-boot:run`). Abrir http://localhost:8080.
4. Ejecutar las pruebas con Test (o `mvn test`).